Thread Communication – Bank Account

Implement a thread-safe BankAccount class that maintains a balance. Multiple Depositor threads must perform deposit operations, and multiple Withdrawer threads must perform withdrawal operations.

CONSTRAINTS AND REQUIREMENTS

1. The balance must never go negative.

- If a withdraw operation would overdraw the account, the withdrawer thread must wait until enough funds are available.

2. The account may optionally define a MAX_BALANCE.

- If a deposit would exceed this limit, the depositor thread must wait until the balance is lower.

3. The system must run until a global counter TOTAL_TRANSACTIONS (successful deposits + successful withdrawals) is reached.

- After this number of successful operations, all threads must cleanly stop.

4. Implement the following methods inside BankAccount:

synchronized void deposit(int amount)

synchronized void withdraw(int amount)

- Both must use a while loop before calling wait().

- Use notifyAll() when the balance changes.

5. Use a thread-safe global counter (e.g., AtomicInteger) to track the number of successful transactions.

6. Depositor and Withdrawer threads must pick random amounts within given ranges (e.g., deposits 10–100, withdrawals 10–80).

7. Each successful transaction must be logged with:

   Thread name

   Operation type (Deposit/Withdraw)

   Amount

   Updated balance

8. The solution must avoid busy waiting.

   Thread.sleep() may only be used to slow down output for readability.

9. After reaching TOTAL_TRANSACTIONS, the system must:

   - Stop all producer/consumer threads cleanly

   - Main thread must join() all child threads